

CELPIPACE.CA

Database Audit Report

Functional Testing of User-Scoped DB Operations

Supabase / Postgres · React + Vercel API

Generated April 2026

1. Executive Summary

This report covers every database operation tied to an individual user in the CELPIPACE web application. It maps each user-facing action to its corresponding write/read on Supabase tables, validates RLS enforcement, identifies error-handling gaps, and surfaces operational risks.

18
TABLES AUDITED

30+
DB OPERATIONS

10
RPC FUNCTIONS

5
CRITICAL RISKS

Contents

1. Executive Summary

2. Database Schema (per table)

3. DB Operations by Code Location (hooks, pages, API)

4. RPC Functions

5. RLS Policy Matrix

6. User Journey → Data Flow

7. Risks & Gaps (prioritised)

8. Recommendations

2. Database Schema

2.1 Auth & Profile

profiles — user account & subscription state

Column group	Fields
Identity	id (PK, FK auth.users), email, full_name, avatar_url
Subscription	is_premium, subscription_status, current_plan, premium_source, premium_granted_at, premium_expires_at, cancel_at_period_end, current_period_start, current_period_end, last_payment_at
Stripe	stripe_customer_id, stripe_subscription_id
Activity	last_seen_at, created_at, updated_at

RLS: users read/insert/update own row (safe columns only). Trigger `guard_profile_billing_columns()` rejects client writes to billing fields. Admin (`sales@celpipace.com`) has full read/update.

2.2 Practice & Test Sessions

Table	Purpose	User-scoped?	Append-only?
<code>practice_sets</code>	Question pool (sections, parts, sets)	No (public)	n/a
<code>questions</code>	MCQ items linked to sets	No (public)	n/a
<code>user_attempts</code>	Per-question selected answers	Yes (FK user_id)	No (update/delete allowed)
<code>test_sessions</code>	Resumable session state (answers, scores, meta JSONB)	Yes (FK user_id)	No (debounced updates)
<code>practice_attempts</code>	Score history per completion	Yes (FK user_id)	Yes
<code>user_progress</code>	Cloud mirror of localStorage progress JSON	Yes (PK user_id)	No (upsert)

2.3 Writing Practice

Table	Purpose	User-scoped?
<code>writing_sets</code>	20 writing sets	No
<code>writing_tasks</code>	Email + survey tasks per set	No
<code>writing_responses</code>	User submission text + word_count + time_spent	Yes (UNIQUE user_id, task_id)

2.4 Subscription & Payments

Table	Purpose	Writer
<code>payments</code>	Revenue log per Stripe charge	Service role (webhook)
<code>coupons</code>	Promo codes (admin-managed)	Admin
<code>coupon_redemptions</code>	Per-user redemption history	RPC <code>redeem_coupon</code>
<code>subscription_events</code>	State-change audit (checkout, cancel, expired, past_due)	Service role + cancel API
<code>cancellation_feedback</code>	Reason / free_text / would_return	Cancel API
<code>webhook_events</code>	Raw Stripe event archive	Webhook handler
<code>email_log</code>	Transactional email audit (welcome, receipt, cancel, past_due)	sendEmail() helper
<code>contact_messages</code>	Support form submissions	Anyone (RLS allows nullable user_id)

3. DB Operations by Code Location

3.1 Hooks

useProgress.js — localStorage + cloud progress

Op	Table	Trigger	RLS	Error handling
SELECT	user_progress	Auth state change (sign-in / token refresh)	RLS	Silent .catch — no UI
UPSERT	user_progress	After local merge with cloud	RLS	Silent .catch
INSERT	practice_attempts	recordCompletion() — user finishes a set	RLS	console.warn only, fire-and-forget

useTestSession.js — resumable sessions

Op	Target	Trigger	RLS	Notes
RPC	start_or_resume_practice_session	Hook bootstrap (kind=practice)	SECURITY DEFINER	console.warn on failure
RPC	start_or_resume_mock_session	Hook bootstrap (kind=mock)	SECURITY DEFINER	console.warn
UPDATE	test_sessions	Debounced 200–400 ms after answer save / nav	RLS	setSaveError; flush() on pagehide
RPC	complete_test_session	User finishes test or reset()	SECURITY DEFINER	console.warn

usePracticeSet.js — question fetcher

Single SELECT on `questions` table by `(section, part)`. Public read, no RLS scoping.

3.2 API Endpoints (service-role)

api/stripe-webhook.js

Stripe event	DB writes	Email
<code>checkout.session.completed</code>	UPSERT profiles, UPSERT payments, INSERT subscription_events, UPSERT webhook_events	welcome
<code>customer.subscription.updated</code>	UPDATE profiles, INSERT subscription_events	—
<code>customer.subscription.deleted</code>	UPDATE profiles (is_premium=false), INSERT subscription_events	cancel_final
<code>invoice.paid</code>	UPDATE profiles (extend expiry), UPSERT payments	receipt
<code>invoice.payment_failed</code>	UPDATE profiles (status=past_due)	past_due
<code>charge.refunded</code>	UPDATE payments (status=refunded)	—
(every event)	UPSERT webhook_events archive, RPC expire_premium_users (backstop)	—

api/cancel-subscription.js

- 1. Auth bearer token validated → SELECT profile.
- 2. Stripe API: set `cancel_at_period_end=true` .
- 3. UPDATE profiles (cancel_at_period_end, period_end, premium_expires_at).
- 4. INSERT cancellation_feedback (**not awaited**).
- 5. INSERT subscription_events — event_type `user.cancel_requested` (**not awaited**).
- 6. sendEmail cancel_scheduled (**not awaited**).

api/contact.js

INSERT `contact_messages` → send email via Resend → UPDATE row with `email_sent` / `email_error` . Returns 202 if email fails.

api/cron-sweep.js

Vercel Cron (daily) → RPC `expire_premium_users()` . Auth via `CRON_SECRET` header.

api/create-checkout-session.js / customer-portal.js

Stripe API only; no direct DB writes (state changes flow back through webhook).

3.3 Pages with Direct DB Calls

Page	Operation	Notes
WritingPracticePage.jsx	SELECT writing_tasks → UPSERT writing_responses	UPSERT keyed on (user_id, task_id) — one submission per task
AuthContext.jsx	RPC touch_user_activity (5-min throttle); UPSERT profiles fallback	Updates last_seen_at
AdminPage.jsx	RPC get_admin_user_activity, UPDATE profiles (toggle premium), INSERT/UPDATE coupons, SELECT subscription_events / cancellation_feedback / email_log	Uses <code>adminSupabase</code> with <code>SUPABASE_SERVICE_ROLE_KEY</code> ; admin email gate

4. RPC Functions

Function	Caller	Security	Effect
<code>start_or_resume_practice_session</code>	useTestSession	SECURITY DEFINER, auth.uid()	Find or create active practice test_sessions row
<code>start_or_resume_mock_session</code>	useTestSession	SECURITY DEFINER	Find or create active mock test_sessions row
<code>complete_test_session</code>	useTestSession	SECURITY DEFINER	Mark is_completed=true

Function	Caller	Security	Effect
<code>redeem_coupon(code)</code>	UpgradeModal / Pricing	SECURITY DEFINER	Validates code, INSERT coupon_redemptions, UPDATE profiles premium
<code>am_i_premium()</code>	AuthContext	SECURITY DEFINER	Wrapper for <code>is_premium_active(auth.uid())</code>
<code>assert_premium()</code>	Server-side guards	SECURITY DEFINER	Raises exception if not premium
<code>expire_premium_users()</code>	cron-sweep, webhook backstop	SECURITY DEFINER (service role)	Bulk UPDATE stale premium rows
<code>touch_user_activity()</code>	AuthContext	SECURITY DEFINER	UPDATE last_seen_at
<code>get_admin_user_activity()</code>	AdminPage	SECURITY DEFINER, admin gate	Joined profile + activity counts

5. RLS Policy Matrix

Table	User read	User insert	User update	User delete	Admin
profiles	own	own (safe)	own (safe)	—	full
practice_sets	public	—	—	—	full
questions	public	—	—	—	full
user_attempts	own	own	own	own	—
test_sessions	own	own	own	own	read
practice_attempts	own	own	—	—	read
user_progress	own	own	own	—	read
writing_sets / writing_tasks	public	—	—	—	full
writing_responses	own	own	own	—	read
payments	own	service-role	service-role	—	read
coupons	—	—	—	—	full
coupon_redemptions	own	RPC	—	—	read
subscription_events	own	service-role	—	—	read
cancellation_feedback	own	own/service	—	—	read
email_log	own	service-role	service-role	—	read
webhook_events	—	service-role	service-role	—	read
contact_messages	own	anyone (incl. anon)	—	—	read

6. User Journey → Data Flow

Journey A — Sign Up → First Practice

1. auth.users created → trigger inserts profile row.
2. Practice completion → localStorage updated → async INSERT practice_attempts (**fire-and-forget**).
3. RPC touch_user_activity refreshes last_seen_at.

Journey B — Test Session Resume

1. RPC start_or_resume_practice_session loads prior selected_answers / scores.
2. Each answer/nav debounces UPDATE test_sessions (200–400 ms).
3. pagehide / visibilitychange → flush().
4. Finish → RPC complete_test_session.

Journey C — Writing Submission

1. SELECT writing_tasks by (set_id, task_number).
2. UPSERT writing_responses on (user_id, task_id) — immutable per task.

Journey D — Premium Purchase

1. POST /api/create-checkout-session → Stripe redirect.
2. checkout.session.completed webhook:
 - UPSERT profiles (is_premium, expires, customer_id, sub_id)
 - UPSERT payments (revenue audit)
 - INSERT subscription_events
 - sendEmail welcome → INSERT/UPDATE email_log

Journey E — Subscription Cancellation

1. POST /api/cancel-subscription with bearer token.
2. Stripe: cancel_at_period_end=true.
3. UPDATE profiles (cancel flag, period_end).
4. INSERT cancellation_feedback & subscription_events (**not awaited**).
5. sendEmail cancel_scheduled.
6. At period end: customer.subscription.deleted → UPDATE profiles is_premium=false → sendEmail cancel_final.

7. Risks & Gaps

CRITICAL — Practice score INSERT is fire-and-forget

In `useProgress.js`, `saveAttemptToCloud()` calls Supabase INSERT without await; failures only `console.warn`. A user's score may silently fail to persist to `practice_attempts` with no UI indication.

CRITICAL — Profile lookup in webhook can produce orphan payments

Stripe webhook resolves user via email/metadata. If the email mismatches the profile, INSERT into `payments` proceeds with `user_id=NULL` (FK is ON DELETE SET NULL). Audit becomes hard.

HIGH — Cancellation side-effects not awaited

`cancel-subscription.js` inserts into `cancellation_feedback` and `subscription_events` with `.then().catch()`. Failures are silent; the user receives 200 even if audit data is lost.

HIGH — Debounced session updates can lose final answers

`useTestSession` defaults to 200–400 ms debounce. On mobile, a pagehide may not fire reliably; the last answer may not flush.

MEDIUM — Renewal payment INSERT errors not checked

`invoice.paid` handler upserts payments without inspecting `error`. Constraint violations or schema drift go unnoticed.

MEDIUM — No rate limiting on public endpoints

`contact.js`, `cancel-subscription.js`, `create-checkout-session.js` are open to repeated invocation by authenticated or unauthenticated callers.

MEDIUM — `user_progress` conflict resolution is last-write-wins

Two-device usage can lose one device's recent progress. Consider merging by per-key `updated_at`.

VERIFIED OK — All user-scoped tables have RLS enabled

`profiles` (with billing trigger), `test_sessions`, `practice_attempts`, `user_progress`, `writing_responses`, `subscription_events`, `cancellation_feedback`, `email_log`, `payments` — each enforces `user_id = auth.uid()` on read paths.

VERIFIED OK — Service-role key never reaches the browser

`adminSupabase` and `api/*` use `SUPABASE_SERVICE_ROLE_KEY` only on the server; client uses anon key.

8. Recommendations

Immediate (this sprint)

1. Await `saveAttemptToCloud()` in `useProgress.js`; show a toast on failure and retry once with backoff.
2. In `api/cancel-subscription.js`, await both audit INSERTs & collect errors into the response (or `Promise.allSettled` with sentry log).

3. In Stripe webhook, abort INSERT into `payments` if profile lookup fails — surface a row in `webhook_events.processing_error` instead.
4. Check `error` on every supabase call inside the webhook; bubble up to `processing_error`.

Short-term

1. Add rate limiting (e.g. Upstash Redis) to `/api/contact`, `/api/cancel-subscription`.
2. Replace fire-and-forget email sends with a queue (Inngest / Trigger.dev) for retries.
3. Add `updated_at`-based merge in `user_progress` sync to handle multi-device.
4. Add a Sentry/log line for any debounced `flush()` that errors so we see lost-data incidents.

Medium-term

1. Add a `profile_billing_audit` table written by the guard trigger when blocked.
2. Introduce idempotency keys on `/api/cancel-subscription` to avoid duplicate flips.
3. Backfill `user_id` on any historical NULL rows in `payments` via Stripe `customer_id` lookup.